Week 36: The Bias–Variance Tradeoff and Resampling Methods

Why the test error is U-shaped, and how to estimate it honestly

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 36

Monday lecture (2×45 min):

- A lean reminder of OLS, Ridge and Lasso from last week (∼10–15 min)
- The training/test figure as function of polynomial degree – the question this whole week answers
- The statistical toolbox (Sections 2.3–2.8): expectation values, sample estimators, the central limit theorem, and the statistics of the OLS and Ridge estimators
- Deriving the bias-variance tradeoff, Eq. (2.47), with code
- Resampling methods: the bootstrap and k-fold cross-validation

Tuesday session (2×45 min, flipped): your questions, then two cases – the bias-variance decomposition by the bootstrap, and cross-validation for the Ridge penalty. Bring your laptop.

Reading

Chapter 2, Sections 2.3–2.12 of the book. Hastie et al., Sections 7.1–7.5, 7.10 (cross-validation) and 7.11 (bootstrap). Every few slides there is a *Pause and think* frame: talk to your neighbour for a minute, then we discuss.

Reminder from week 35: the three estimators in one view

With the SVD $\mathsf{X} = \mathsf{U}\mathbf{\Sigma}\mathsf{V}^T$, ordinary least squares reads

$$\hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = \left(\mathsf{X}^T\mathsf{X}\right)^{-1}\mathsf{X}^T\mathsf{y} = \sum_{i=0}^{p-1} \frac{\mathsf{u}_i^T\mathsf{y}}{\sigma_i}\,\mathsf{v}_i, \qquad \tilde{\mathsf{y}}_{\mathrm{OLS}} = \mathsf{U}\mathsf{U}^T\mathsf{y},$$

Ridge regression changes the picture in exactly one place, multiplying each mode by the shrinkage factor of Eq. (3.48),

$$\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}} = \sum_{i=0}^{p-1} \frac{\sigma_i^2}{\sigma_i^2+\lambda}\,\frac{\mathsf{u}_i^T\mathsf{y}}{\sigma_i}\,\mathsf{v}_i,$$

and the Lasso replaces smooth shrinkage by soft thresholding, Eq. (3.63): coefficients are moved towards zero by $\lambda/2$ and clipped at zero – shrinkage *and* selection.

Last week's language

We said: "Ridge trades a little bias for a large reduction in variance." We used the words – today we earn them. Bias and variance *of what*, with respect to *which* randomness?

Reminder from week 35: what the penalty does to the numbers

For a singular value σ_i of the design matrix:

- **OLS** keeps the mode untouched: factor 1. Small σ_i (nearly collinear columns) amplify the data noise by $1/\sigma_i$.
- **Ridge** damps each mode by $\sigma_i^2/(\sigma_i^2+\lambda)$: modes with $\sigma_i^2 \gg \lambda$ survive, modes with $\sigma_i^2 \ll \lambda$ are suppressed.
- **Lasso** sets small coefficients exactly to zero (the corner of the 1-norm ball from last week's geometry figure).

This week we make the statistical content of these statements precise, and we meet the experimental tools – resampling methods – that let us *measure* bias and variance from a single data set.

Pause and think 1: which estimator is unbiased?

Question

One of the three estimators above satisfies $\mathbb{E}[\hat{\boldsymbol{\theta}}] = \boldsymbol{\theta}$, the true parameters. Which one, and what assumption about the data does the statement silently rely on?

Pause and think 1: which estimator is unbiased?

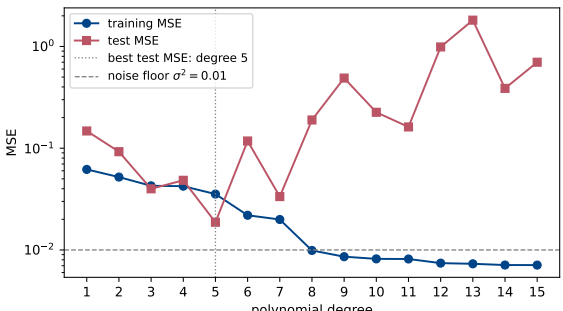
Question

One of the three estimators above satisfies $\mathbb{E}[\hat{\boldsymbol{\theta}}] = \boldsymbol{\theta}$, the true parameters. Which one, and what assumption about the data does the statement silently rely on?

Answer

OLS, Eq. (2.40) – *provided* the data really come from $\mathrm{y} = \mathrm{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ with zero-mean noise. If the true $f(\mathrm{x})$ is not in the span of our features, OLS is biased no matter how much data we collect. Ridge is biased for any $\lambda > 0$: $\mathbb{E}[\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}}] = (\mathrm{X}^T\mathrm{X} + \lambda \mathrm{I})^{-1}\mathrm{X}^T\mathrm{X}\,\boldsymbol{\theta} \neq \boldsymbol{\theta}$.

The figure that drives this week



Training and test MSE against polynomial degree, from last week's exercise 3 (seed 2026, $n = 100$).

- The **training error** falls monotonically – a degree-$(n-1)$ polynomial would drive it to zero.
- The **test error** falls, reaches a broad minimum, and rises: the celebrated U-curve.

This week's question: what law of statistics forces the U-shape, and how do we find its minimum when data are scarce?

The answer is an identity, Eq. (2.47), plus two experimental techniques: the bootstrap and cross-validation. To derive the identity we first need the statistical toolbox of Sections 2.3–2.8.

Toolbox 1: expectation values, moments and variance (Section 2.3)

For a stochastic variable X with PDF $p(x)$ and a function h,

$$\mathbb{E}[h] = \int h(x)\, p(x)\, dx \qquad \text{(Eq. 2.12; a sum in the discrete case).}$$

The moments $\mathbb{E}[x^n]$ summarise p; the two we use constantly are the **mean** $\mu = \mathbb{E}[x]$ and the **variance**, Eq. (2.16),

$$\sigma_X^2 = \mathrm{Var}(X) = \mathbb{E}\left[(x-\mu)^2\right] = \mathbb{E}[x^2] - (\mathbb{E}[x])^2\,,$$

– the mean of the square minus the square of the mean, worth memorising. From linearity, Eq. (2.17),

$$\mathbb{E}[aX+b] = a\,\mathbb{E}[X] + b, \qquad \mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)\,:$$

the variance ignores shifts and scales *quadratically* – the reason a feature in millimetres has 10^6 times the variance of the same feature in metres, and the reason scaling mattered so much last Tuesday.

Toolbox 2: covariance and independence (Section 2.4)

For two stochastic variables, Eqs. (2.18) and (2.19),

$$\mathrm{Cov}(X_i, X_j) = \mathbb{E}\left[(x_i - \mu_i)(x_j - \mu_j)\right] = \mathbb{E}[x_i x_j] - \mathbb{E}[x_i]\,\mathbb{E}[x_j],$$

with $\mathrm{Cov}(X_i, X_i) = \mathrm{Var}(X_i)$. If X_i and X_j are **independent**, the joint PDF factorises and the covariance vanishes, Eq. (2.20).

The converse is false, and it matters

Take X symmetric about zero and $Y = X^2$: then $\mathrm{Cov}(X, Y) = 0$ although Y is a function of X. Covariance sees only *linear* dependence.

Independence is the assumption hiding inside almost everything today: the central limit theorem, the bootstrap and the random splits of cross-validation all require it. Keep asking: *are my samples independent?*

Toolbox 3: samples and estimators (Section 2.6)

In practice we never know $p(x)$; we have n measurements. From them we build the **sample mean** and **sample variance**, Eq. (2.25),

$$\bar{x} = \frac{1}{n}\sum_{k=0}^{n-1} x_k, \qquad s^2 = \frac{1}{n-1}\sum_{k=0}^{n-1}(x_k - \bar{x})^2.$$

These are **estimators**: functions of the data that approximate properties of the unknown distribution.

The one idea to take home today

An estimator is a function of random variables, so it is *itself a random variable*, with a distribution, a mean and a variance of its own. Everything that follows – the bias-variance tradeoff, the bootstrap, cross-validation – is about exploring that distribution.

Pause and think 2: why $n-1$?

Question

The sample variance in Eq. (2.25) divides by $n-1$, yet the sample mean divides by n. Why the asymmetry?

Pause and think 2: why $n-1$?

Question

The sample variance in Eq. (2.25) divides by $n-1$, yet the sample mean divides by n. Why the asymmetry?

Answer

Using $\bar{x}$ instead of the unknown μ makes the squared deviations systematically too small: $\bar{x}$ is precisely the value that *minimises* $\sum_k (x_k - c)^2$ for the sample at hand. A short calculation, Eq. (2.28), gives $\mathbb{E}[\frac{1}{n}\sum_k (x_k - \bar{x})^2] = \frac{n-1}{n}\sigma^2$: one degree of freedom was spent estimating the mean, and dividing by $n-1$ (Bessel's correction) restores unbiasedness. Same story behind the $n-p$ in the residual variance of regression.

Toolbox 4: bias, variance and the MSE of an estimator (Section 2.6)

Let $\hat{\theta}$ estimate a quantity θ. Its **bias** is the systematic error, Eq. (2.26), $\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$; its **variance** measures how much it moves when the sample is redrawn. Adding and subtracting $\mathbb{E}[\hat{\theta}]$ in the mean squared error gives Eq. (2.27),

$$\mathbb{E}\left[(\hat{\theta} - \theta)^2\right] = \underbrace{\left(\mathbb{E}[\hat{\theta}] - \theta\right)^2}_{\mathrm{Bias}^2} + \underbrace{\mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right]}_{\mathrm{Var}(\hat{\theta})}$$

the cross term vanishing because $\mathbb{E}\left[\hat{\theta} - \mathbb{E}[\hat{\theta}]\right] = 0$.

Why this innocent identity runs the whole week

An unbiased estimator is not automatically a good one: a biased estimator with much smaller variance can have the *smaller total error*. That is the bargain Ridge offers. The bias-variance tradeoff of Section 2.10 is this identity applied to a *prediction* instead of a parameter.

Toolbox 5: the central limit theorem (Section 2.5)

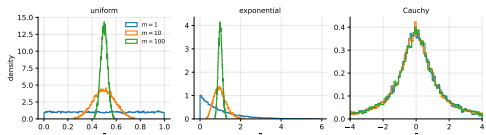
Average m independent draws from *any* PDF $p(x)$ with mean μ and finite variance σ^2, Eq. (2.22):

$$z = \frac{x_0 + x_1 + \cdots + x_{m-1}}{m}.$$

Then as m grows, $\tilde{p}(z)$ tends to a Gaussian with mean μ and variance σ^2/m, whatever the shape of $p(x)$. The practical statement is the **standard error**, Eq. (2.24),

$$\boxed{\sigma_m = \frac{\sigma}{\sqrt{m}}}$$

Four times the data halves the uncertainty: large data sets help, and help *slowly*.



Uniform and exponential averages become Gaussian by $m \sim 10$; Cauchy averages never do – no finite variance, no theorem.

What the CLT promises – and what it does not

Two conditions, both routinely forgotten:

1. **Finite variance.** Heavy-tailed distributions (Cauchy) have none; their sample means never settle down.
2. **Independence.** For correlated data the standard error $\sigma/\sqrt{m}$ *underestimates* the true uncertainty, often by a large factor (Section 2.7: the factor is the autocorrelation time τ, and methods like blocking exist to repair it).

Machine learning connection

Eq. (2.24) is why a test set of 1000 points pins the test error to roughly 3% relative accuracy – and why comparing two models whose test errors differ by less than that is meaningless. It also underwrites the bootstrap below, and explains why the gradient noise in stochastic gradient descent scales as $1/\sqrt{B}$ with batch size B.

Pause and think 3: error bars on the test error

Question

You compare two regression models on the same test set of $n = 1000$ points and find test MSEs of 0.100 and 0.103. Is the first model better?

Pause and think 3: error bars on the test error

Question

You compare two regression models on the same test set of $n = 1000$ points and find test MSEs of 0.100 and 0.103. Is the first model better?

Answer

You cannot tell. The test MSE is itself a sample mean of 1000 squared residuals, so it carries a standard error of order $\sigma_{\text{res}}/\sqrt{1000} \approx 3\%$ of its value – as large as the difference. You would need either a much larger test set or paired comparisons across resampled splits (bootstrap or cross-validation, coming up) before declaring a winner.

The statistics of the least-squares estimator (Section 2.8)

Assume the data are generated by the linear model, Eqs. (2.37) and (2.38),

$$\mathrm{y} = \mathrm{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \text{ independent},$$

so that, Eq. (2.39), $\mathbb{E}[y_i] = \mathrm{X}_{i,*}\boldsymbol{\theta}$ and $\mathrm{Var}(y_i) = \sigma^2$: the design matrix is fixed, all randomness enters through the noise. Then

$$\mathbb{E}[\hat{\boldsymbol{\theta}}] = (\mathrm{X}^T\mathrm{X})^{-1}\mathrm{X}^T\,\mathbb{E}[\mathrm{y}] = \boldsymbol{\theta} \qquad \text{(Eq. 2.40: OLS is unbiased)},$$

and using $\mathbb{E}[\mathrm{y}\mathrm{y}^T] = \mathrm{X}\boldsymbol{\theta}\boldsymbol{\theta}^T\mathrm{X}^T + \sigma^2\mathrm{I}$,

$$\boxed{\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2 \left(\mathrm{X}^T\mathrm{X}\right)^{-1}} \qquad \text{(Eq. 2.42)}.$$

The standard error of coefficient j is $\sigma(\hat{\theta}_j) = \sigma\sqrt{[(\mathrm{X}^T\mathrm{X})^{-1}]_{jj}}$, Eq. (2.43) – exactly what you need for the confidence intervals of this week's exercises, with $\hat{\sigma}^2 = \|\mathrm{y} - \mathrm{X}\hat{\boldsymbol{\theta}}\|_2^2/(n-p)$.

The same statistics for Ridge

Repeating the computation for the Ridge estimator gives

$$\mathbb{E}\big[\hat{\boldsymbol{\theta}}_{\text{Ridge}}\big] = (\mathsf{X}^T\mathsf{X} + \lambda \mathsf{I}_{pp})^{-1}(\mathsf{X}^T\mathsf{X})\,\boldsymbol{\theta} \neq \boldsymbol{\theta} \quad (\lambda > 0): \qquad \text{biased},$$

$$\operatorname{Var}\big[\hat{\boldsymbol{\theta}}_{\text{Ridge}}\big] = \sigma^2\,[\mathsf{X}^T\mathsf{X} + \lambda \mathsf{I}]^{-1}\mathsf{X}^T\mathsf{X}\,\big([\mathsf{X}^T\mathsf{X} + \lambda \mathsf{I}]^{-1}\big)^T \xrightarrow{\lambda \to \infty} 0.$$

The difference is non-negative definite,

$$\operatorname{Var}[\hat{\boldsymbol{\theta}}_{\text{OLS}}] - \operatorname{Var}[\hat{\boldsymbol{\theta}}_{\text{Ridge}}] = \sigma^2[\mathsf{X}^T\mathsf{X} + \lambda \mathsf{I}]^{-1}\left[2\lambda \mathsf{I} + \lambda^2(\mathsf{X}^T\mathsf{X})^{-1}\right]\big([\mathsf{X}^T\mathsf{X} + \lambda \mathsf{I}]^{-1}\big)^T \succeq 0,$$

so **for every** $\lambda > 0$ Ridge has strictly smaller variance than OLS. Whether the trade is worth it is exactly the question Eq. (2.27) answers: it is, whenever the squared bias introduced is smaller than the variance removed.

Pause and think 4: read Eq. (2.42) through the SVD

Question

With $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ we have $(\mathbf{X}^T\mathbf{X})^{-1} = \mathbf{V}\tilde{\mathbf{\Sigma}}^{-2}\mathbf{V}^T$, so the variance of $\hat{\boldsymbol{\theta}}$ along singular direction i is σ^2/σ_i^2. What does a small singular value do, and what does Ridge do about it?

Pause and think 4: read Eq. (2.42) through the SVD

Question

With $\mathrm{X} = \mathrm{U}\Sigma\mathrm{V}^T$ we have $(\mathrm{X}^T\mathrm{X})^{-1} = \mathrm{V}\tilde{\Sigma}^{-2}\mathrm{V}^T$, so the variance of $\hat{\boldsymbol{\theta}}$ along singular direction i is σ^2/σ_i^2. What does a small singular value do, and what does Ridge do about it?

Answer

A nearly collinear pair of features means a small σ_i and an *enormous* variance σ^2/σ_i^2 in that direction: the data cannot determine the parameter. The numerical ill-conditioning of week 34 and the statistical variance blow-up are the same fact in two languages. Ridge multiplies each mode by $\sigma_i^2/(\sigma_i^2 + \lambda)$, taming exactly the directions that misbehave – the promised variance reduction, mode by mode.

Where does the squared cost come from? OLS as maximum likelihood

With Gaussian noise, a single output is distributed as

$$p(y_i, \mathrm{X}|\boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \mathrm{X}_{i,*}\boldsymbol{\theta})^2}{2\sigma^2}\right],$$

and for independent, identically distributed events the likelihood of the whole data set D is the product

$$p(\mathrm{D}|\boldsymbol{\theta}) = \prod_{i=0}^{n-1} p(y_i, \mathrm{X}|\boldsymbol{\theta}).$$

Maximum likelihood: choose $\boldsymbol{\theta}$ so that the observed data are the most probable. Products of small numbers underflow and are painful to differentiate, so we minimise the *negative log-likelihood* instead – the logarithm is monotonic, the optimum does not move.

The negative log-likelihood is the OLS cost

$$C(\boldsymbol{\theta}) = -\log p(\mathsf{D}|\boldsymbol{\theta}) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{\|\mathsf{y} - \mathsf{X}\boldsymbol{\theta}\|_2^2}{2\sigma^2}.$$

The first term is a constant; minimising C is minimising the mean squared error. Setting the gradient to zero,

$$\mathsf{X}^T(\mathsf{y} - \mathsf{X}\boldsymbol{\theta}) = 0 \quad \Longrightarrow \quad \hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = (\mathsf{X}^T\mathsf{X})^{-1}\mathsf{X}^T\mathsf{y}.$$

Why show this now

The squared cost is not an aesthetic choice: it *is* the Gaussian noise assumption. Next week we put a prior on $\boldsymbol{\theta}$ and rerun the same argument – a Gaussian prior will hand us Ridge, a Laplace prior the Lasso.

The bias-variance tradeoff: the setup (Section 2.10)

The data are generated by a noisy process, Eq. (2.44),

$$\mathrm{y} = f(\mathrm{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

with f unknown. We fit a model on a training set and obtain predictions $\tilde{\mathrm{y}}$; the cost we decompose is Eq. (2.45),

$$\mathbb{E}\left[(\mathrm{y} - \tilde{\mathrm{y}})^2\right] = \frac{1}{n}\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2.$$

The essential point – and the source of most confusion

The expectation runs over **two** sources of randomness: the noise ε *and the training set itself*. The model $\tilde{\mathrm{y}}$ is a random variable because the data it was fitted to were random: redraw the training set and the fitted polynomial moves. $\mathbb{E}[\tilde{\mathrm{y}}]$ is the *average prediction over training sets*.

The derivation, step by step

Insert $y = f + \varepsilon$ and add and subtract $\mathbb{E}[\tilde{y}]$, Eq. (2.46),

$$\mathbb{E}\left[(y - \tilde{y})^2\right] = \mathbb{E}\left[(f + \varepsilon - \tilde{y} + \mathbb{E}[\tilde{y}] - \mathbb{E}[\tilde{y}])^2\right].$$

Expanding gives three squared terms and three cross terms. Every cross term dies:

- terms linear in ε: the noise has zero mean and is independent of the training set;
- the remaining term: $\mathbb{E}[\tilde{y} - \mathbb{E}[\tilde{y}]] = 0$, while f and $\mathbb{E}[\tilde{y}]$ are not stochastic.

What survives is Eq. (2.47):

$$\mathbb{E}\left[(y - \tilde{y})^2\right] = \underbrace{\frac{1}{n}\sum_i (f_i - \mathbb{E}[\tilde{y}])^2}_{\text{Bias}^2} + \underbrace{\frac{1}{n}\sum_i (\tilde{y}_i - \mathbb{E}[\tilde{y}])^2}_{\text{Var}[\tilde{y}]} + \sigma^2$$

– Eq. (2.27) again, applied to a prediction instead of a parameter.

Reading the three terms

Bias^2 How far the *average* prediction is from the truth: the error built into the model's simplifying assumptions. A straight line through curved data has large bias, and no amount of data removes it. *Decreases* with model complexity.

Variance How much the prediction moves when the training set is redrawn. A flexible model chases the noise of whatever sample it got, so its predictions differ wildly between training sets. *Increases* with model complexity.

σ^2 The irreducible error: no model can predict the noise term. A hard floor under the test error.

The test error is their sum: it falls while the bias falls, then rises when the variance takes over – **the U-curve of the opening figure is Eq. (2.47) in action**. Model selection is the business of locating the minimum.

Pause and think 5: a suspiciously good model

Question

A fellow student reports a test MSE *below* your best estimate of the noise level σ^2. Brilliant model or a bug?

Pause and think 5: a suspiciously good model

Question

A fellow student reports a test MSE *below* your best estimate of the noise level σ^2. Brilliant model or a bug?

Answer

A bug (or leakage). By Eq. (2.47) the expected test error can never lie below σ^2: no model predicts noise. A test error at or under the noise floor means test data leaked into training or preprocessing, the noise estimate is wrong, or the test set is too small for its own error bar (Pause and think 3). Healthy scepticism about "too good" numbers is a transferable skill.

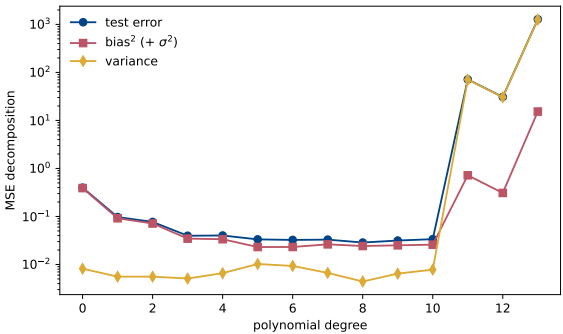
Estimating the three terms: the bootstrap manufactures the ensemble

Eq. (2.47) wants an average over training sets – but we have *one* data set. The bootstrap (details in a moment) fakes the ensemble by resampling the training data with replacement:

```
np.random.seed(2026)
n, n_bootstraps, maxdegree = 40, 100, 14
x = np.linspace(-3, 3, n).reshape(-1, 1)
y = np.exp(-x**2) + 1.5*np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.shape)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    random_state=2026)
for degree in range(maxdegree):
    model = make_pipeline(PolynomialFeatures(degree=degree),
                          LinearRegression(fit_intercept=False))
    y_pred = np.empty((y_test.shape[0], n_bootstraps))
    for i in range(n_bootstraps):
        x_, y_ = resample(x_train, y_train) # draw with replacement
        y_pred[:, i] = model.fit(x_, y_).predict(x_test).ravel()
    error[degree] = np.mean(np.mean((y_test - y_pred)**2, axis=1, keepdims=True))
    bias[degree] = np.mean((y_test - np.mean(y_pred, axis=1, keepdims=True))**2)
    variance[degree] = np.mean(np.var(y_pred, axis=1, keepdims=True))
```

Averages along `axis=1` are expectations over training sets; `keepdims=True` keeps the column shape and is essential for the bias to come out right (drop it and enjoy the debugging).

The decomposition, measured



Seed 2026, $n = 40$, 100 bootstraps (Tuesday's Case 1).

- Bias^2 falls from 0.39 at degree 0 to ≈ 0.02 by degree 5 and flattens: the polynomial family is then rich enough.
- Variance is negligible up to degree $\sim$10, then explodes ($0.008 \to 10^3$).
- The sum has a broad minimum around degrees 5–9.

Because we compare with y_{test} rather than the unknown f, the measured "bias" absorbs $\sigma^2 = 0.01$ – its floor sits just above the true squared bias.

Pause and think 6: equality or inequality?

Question

The code prints `error >= bias + variance`. Our derivation of Eq. (2.47) produced an *equality*. Who is lying?

Pause and think 6: equality or inequality?

Question

The code prints `error >= bias + variance`. Our derivation of Eq. (2.47) produced an *equality*. Who is lying?

Answer

Nobody. The derivation's bias compares $\mathbb{E}[\tilde{y}]$ with the true f, which the code does not know; it uses $y_{\text{test}} = f + \varepsilon$ instead. The measured bias term therefore contains the irreducible σ^2, and the printed identity holds as an equality only in expectation, with finite-sample noise on top. Knowing exactly *which* quantity your code estimates is half of statistics.

Every regularisation method is a move on this curve

- **Ridge and Lasso** add bias to remove variance – now a theorem (the variance difference above), not a slogan.
- **Early stopping** halts training before the variance grows.
- **Bagging and random forests** average many high-variance models to cancel variance; **boosting** combines many high-bias models to reduce bias.
- **Dropout, data augmentation**: variance reduction in neural networks.

Knowing which of the two terms a method attacks predicts whether it will help.

A modern caveat

The clean U-shape is a statement about models of moderate capacity. Heavily overparameterised networks show *double descent*: past the interpolation threshold the test error can fall again. We return to this in the neural network chapters.

Why resampling methods?

In our eagerness to fit, we omitted the questions that matter:

- How uncertain is an estimated quantity – a parameter, a test error?
- How do we *measure* bias and variance with one finite data set?
- How do we select a model without wasting data?

Resampling: repeatedly draw samples from the training data, refit the model on each, and study how the results vary. Computationally greedy – we fit the same model hundreds of times – and, with today's computers, routinely affordable. The two workhorses of machine learning, both used in Project 1:

1. the **bootstrap** (model assessment: distributions and error bars for estimators);
2. **cross-validation** (model selection: choosing degree, λ, ...).

Relatives: the jackknife (leave-one-out, Section 2.11) and blocking for correlated data (Section 2.7).

The bootstrap: Efron's idea (Section 2.11)

$\hat{\theta} = \hat{\theta}(\mathrm{X})$ is a random variable with some unknown PDF $p(t)$. If we knew the data-generating $p(x)$ we could learn $p(t)$ by brute force: draw fresh samples, recompute $\hat{\theta}$, histogram. We do not know $p(x)$.

Efron (1979)

Replace $p(x)$ by the *empirical* distribution of the observed data. Drawing from it is simply drawing the observed values **with replacement** – and asymptotically this recovers the right answer.

The algorithm:

1. Draw with replacement n values from $\mathrm{x} = (x_0, \ldots, x_{n-1})$, forming the resample x^*.
2. Compute the replica $\hat{\theta}^* = \hat{\theta}(\mathrm{x}^*)$.
3. Repeat k times; the spread of the replicas estimates the distribution of $\hat{\theta}$.

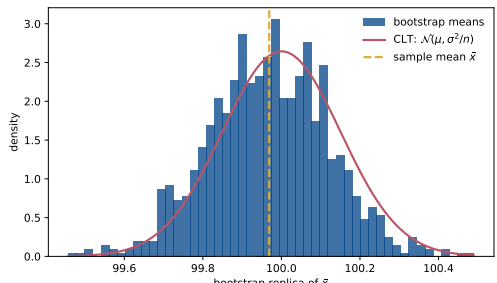
No Gaussian assumption, works for statistics with no known formula (a median, a ratio, a test error), and often beats asymptotic formulae for small samples. It *requires* iid data – for correlated data use block bootstrap.

The bootstrap at work, checked against the CLT

```
def bootstrap(data, statistic, k=1000, rng=None):
    rng = np.random.default_rng() if rng is None
          else rng
    n = len(data)
    replicas = np.empty(k)
    for i in range(k):
        sample = data[rng.integers(0, n, n)]
        replicas[i] = statistic(sample)
    return replicas

rng = np.random.default_rng(2026)
x = rng.normal(100.0, 15.0, 10000)
t = bootstrap(x, np.mean, k=1000, rng=rng)
```

The bootstrap standard error is 0.1523; the CLT prediction $\sigma/\sqrt{n} = 0.1508$. For the mean of iid data the CLT already knew the answer – the value of the bootstrap is that the code keeps working when `np.mean` is replaced by any statistic.



Pause and think 7: why with replacement?

Question

Why must the bootstrap draw *with* replacement? What would k draws *without* replacement of size n give?

Pause and think 7: why with replacement?

Question

Why must the bootstrap draw *with* replacement? What would k draws *without* replacement of size n give?

Answer

Without replacement, every resample of size n is the original data set in a different order: every replica $\hat{\theta}^*$ is identical and the estimated variance is exactly zero. Drawing with replacement makes each resample a genuine draw from the empirical distribution – on average $1 - 1/e \approx 63\%$ of the original points appear, some repeated – so the replicas scatter the way $\hat{\theta}$ would across real repeated experiments.

Cross-validation: structuring the split (Section 2.12)

A single train/test split wastes data, and the answer depends on which points landed where. Random re-splitting helps but leaves some points over-represented. **k-fold cross-validation** removes the imbalance:

1. Shuffle the data set at random.
2. Split it into k roughly equal, mutually exclusive folds.
3. Each fold in turn is the test set; fit on the other $k-1$ folds, evaluate on the held-out fold, record the score, discard the model.
4. Summarise by the mean – and the spread – of the k scores.

Every observation is tested exactly once and trained on exactly $k-1$ times. $k = n$ is **leave-one-out** (LOOCV): the jackknife of Eq. (2.48) applied to prediction error. Unbiased but expensive (n fits) and high-variance (the n training sets are nearly identical). k between 5 and 10 is the usual compromise.

Pause and think 8: choosing k

Question

Your data set has $n = 10^6$ points and one model fit takes a minute. Do you use LOOCV? And on $n = 50$ points?

Pause and think 8: choosing k

Question

Your data set has $n = 10^6$ points and one model fit takes a minute. Do you use LOOCV? And on $n = 50$ points?

Answer

Never at $n = 10^6$: a million fits, and the tiny variance gain is worthless anyway – with that much data even a single split is accurate (CLT again). At $n = 50$, data are precious: LOOCV (50 quick fits) or $k = 10$ is sensible, and the spread across folds gives the error bar the single split could not. The right k is a cost/benefit decision, not a constant of nature.

The standard use: selecting the Ridge penalty

For each λ on a (logarithmic) grid, and each fold i, fit on the data with fold i removed, Eq. (2.49),

$$\boldsymbol{\theta}_{-i}(\lambda) = \left(\mathsf{X}_{-i,*}^{T}\mathsf{X}_{-i,*} + \lambda \mathsf{I}_{pp}\right)^{-1} \mathsf{X}_{-i,*}^{T}\,\mathsf{y}_{-i},$$

evaluate the MSE on fold i, and average over folds. Choose the λ minimising the cross-validated error; refit on all data. The same recipe selects the polynomial degree, the Lasso penalty, a learning rate, a network width – **cross-validation is the universal knob-tuner of this course**. The one iron rule: the test set proper is touched exactly once, at the very end; λ is chosen on the folds, never on the final test data.

Code: own KFold loop vs cross_val_score

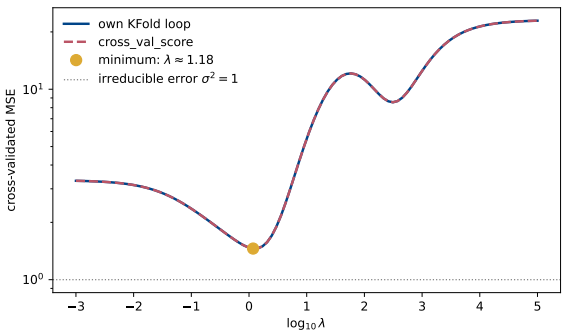
```
rng = np.random.default_rng(2026)
x = rng.standard_normal(100)
y = 3*x**2 + rng.standard_normal(100) # noise variance is 1

lambdas = np.logspace(-3, 5, 100)
kfold = KFold(n_splits=5, shuffle=True, random_state=2026)

# own loop: scaler and Ridge fitted on the training folds only
for i, lmb in enumerate(lambdas):
    for j, (train_inds, test_inds) in enumerate(kfold.split(x)):
        Xtrain = poly.fit_transform(x[train_inds][:, np.newaxis])
        Xtest = poly.transform(x[test_inds][:, np.newaxis])
        scaler = StandardScaler().fit(Xtrain)
        ridge = Ridge(alpha=lmb).fit(scaler.transform(Xtrain), y[train_inds])
        ypred = ridge.predict(scaler.transform(Xtest))
        scores_KFold[i, j] = np.mean((ypred - y[test_inds])**2)

# the same in three lines
pipe = make_pipeline(PolynomialFeatures(degree=6), StandardScaler(), Ridge(alpha=lmb))
scores = cross_val_score(pipe, x[:, np.newaxis], y, cv=kfold,
                         scoring="neg_mean_squared_error")
```

Reading the cross-validation curve



Degree-6 polynomial on $y = 3x^2 + \varepsilon$, $n = 100$, 5 folds (Tuesday's Case 2).

- The two curves lie on top of each other: `cross_val_score` does nothing magical.
- The minimum sits near $\lambda \approx 1$ with CV-MSE ≈ 1.5, hovering just above the irreducible $\sigma^2 = 1$ – the method has recovered the noise floor from 100 points.
- At $\lambda = 10^5$ every coefficient is crushed: the model predicts the mean and the error climbs to $\mathrm{Var}(y)$.

The `StandardScaler` *inside* the fold loop is not decoration: a degree-6 basis has columns differing by orders of magnitude, and without scaling the curve tails are numerical noise.

Pause and think 9: the subtle leak

Question

A pipeline scales the full data set *once*, then runs 5-fold cross-validation on the scaled data. The CV error comes out slightly – but systematically – too optimistic. Why?

Pause and think 9: the subtle leak

Question

A pipeline scales the full data set *once*, then runs 5-fold cross-validation on the scaled data. The CV error comes out slightly – but systematically – too optimistic. Why?

Answer

The scaler's mean and variance were computed on *all* the data, so each training fold has already seen statistics of its own test fold: a leak. Small here, large when features are many. Rule: **every** preprocessing step that learns from data – centring, scaling, feature selection, PCA, imputation – goes *inside* the loop, refitted per fold. In `scikit-learn`: wrap it in a `Pipeline` and cross-validate the pipeline. The same discipline applies to the bootstrap.

Summary of week 36

- Estimators are random variables; their quality is $\mathrm{MSE} = \mathrm{Bias}^2 + \mathrm{Var}$, Eq. (2.27).
- The CLT gives the $\sigma/\sqrt{m}$ standard error, Eq. (2.24) – and error bars on everything, test errors included.
- OLS is unbiased with $\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2(\mathrm{X}^T\mathrm{X})^{-1}$, Eqs. (2.40) and (2.42); Ridge is biased with strictly smaller variance – the tradeoff is now quantitative.
- For predictions: $\mathbb{E}[(\mathrm{y} - \tilde{\mathrm{y}})^2] = \mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$, Eq. (2.47). The U-curve is this identity in action.
- The bootstrap resamples with replacement to estimate distributions of estimators; *k*-fold cross-validation selects models without wasting data. Preprocessing lives inside the folds.

Tomorrow and the deadline

Tuesday: Case 1 = the bias-variance figure, taken apart and stress-tested; Case 2 = cross-validation for λ (and degree). Weekly exercises due **Friday September 4 at midnight**. Next week and week 38: gradient descent methods, with a brief look at the Bayesian reading of OLS, Ridge and Lasso.

Pause and think 10: which tool for which question?

Question

You must (a) put an error bar on the fitted $\hat{\theta}_3$ of a Lasso model, and (b) choose its penalty λ. Bootstrap or cross-validation – which for which, and why not the formula Eq. (2.43)?

Pause and think 10: which tool for which question?

Question

You must (a) put an error bar on the fitted $\hat{\theta}_3$ of a Lasso model, and (b) choose its penalty λ. Bootstrap or cross-validation – which for which, and why not the formula Eq. (2.43)?

Answer

(a) Bootstrap: it estimates the sampling distribution of *any* statistic, and no closed formula like Eq. (2.43) exists for the Lasso (the soft threshold is nonlinear in y). (b) Cross-validation: choosing λ is model selection, a prediction-error question. One sentence to remember: *the bootstrap asks how uncertain my estimate is; cross-validation asks how well my model predicts.*